Design e Arquitectura de Software

2024/2025

Lab. Classes - 6

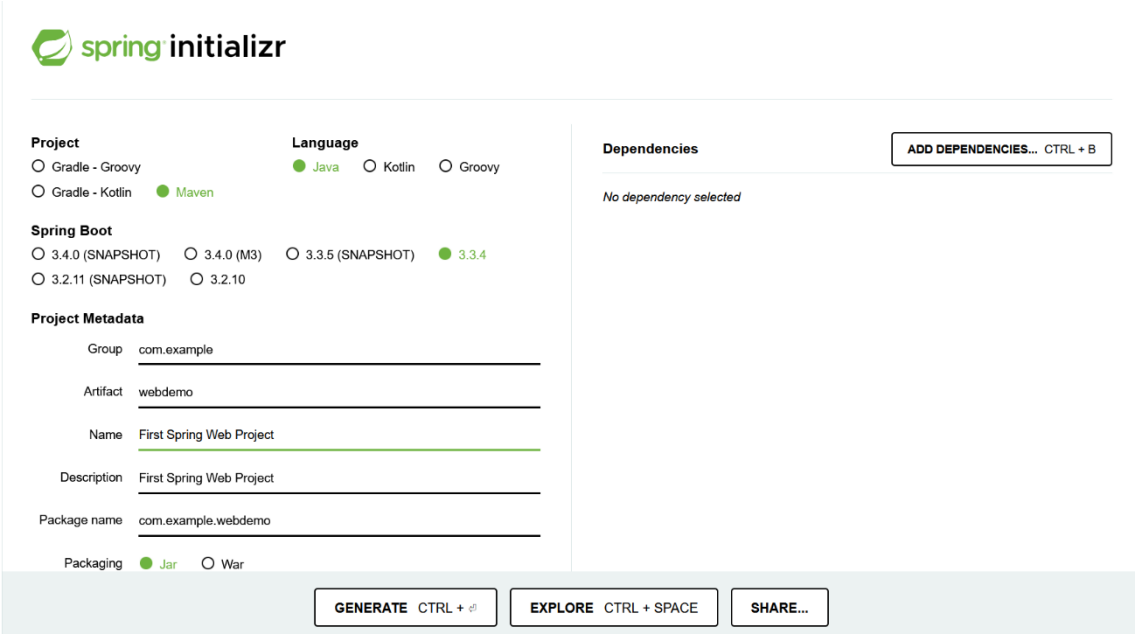
Java web services with Spring-Boot

Spring-boot is a framework for creating enterprise Java applications, offering many convenient features for managing different aspects such as web or persistence. The purpose of this class is just to get started and explore this framework for creating a few simple Web Services. This is loosely based on the tutorial found at:

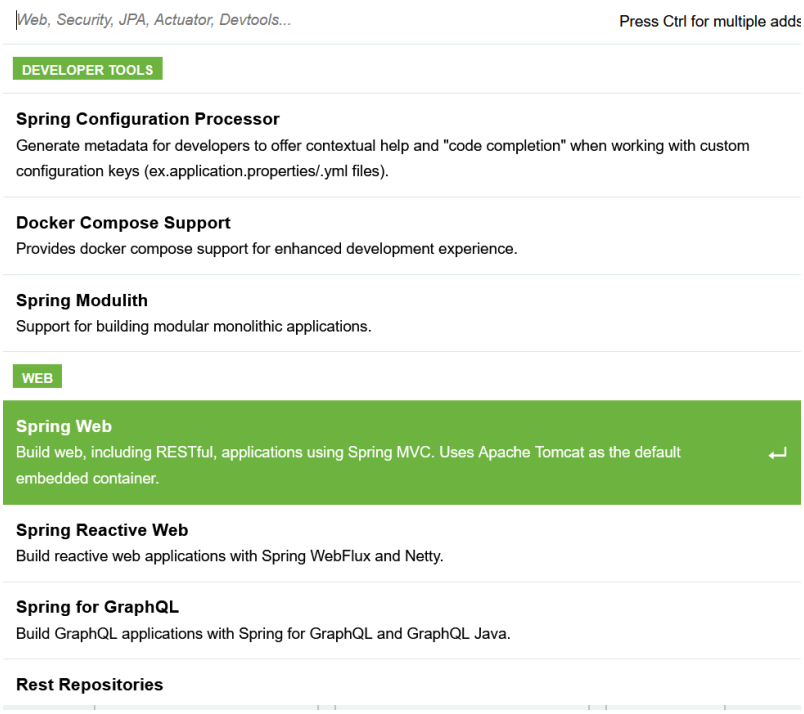
https://spring.io/guides/gs/rest-service#initial

1 – Spring projects can be quickly created using the Spring Initializr (https://start.spring.io/). Using this site, you can configure and download a pre-configured spring-boot project using selected features, language, language version and build system. We will create a simple project including support for Web features. While some IDEs embed support for using the Spring Initialzr directly, we will not rely on this feature and create the project externally, which will then imported into our selected IDE of choice.

We begin creating our first project by selecting the appropriate options in the main screen.

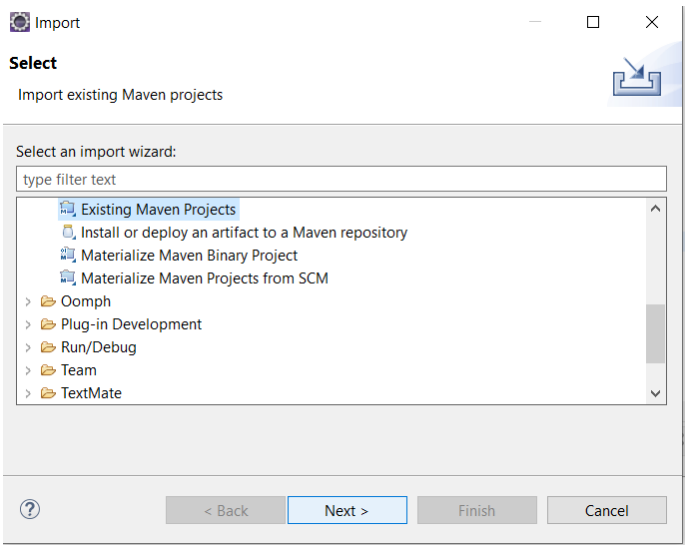


Because we are interested in developing web services, we will add the required dependencies now. Press CTRL+B or click the ADD DEPENDENCIES button and select the Spring Web dependency.



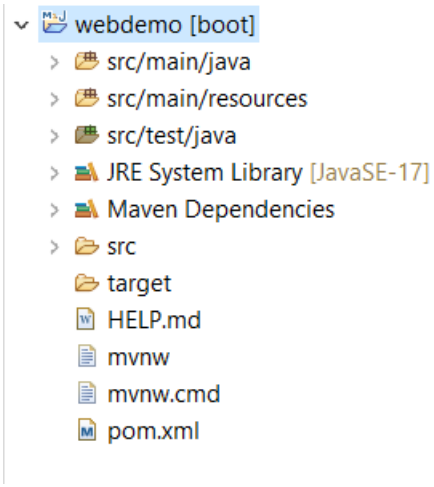
After adding the dependency, press the GENERATE button and a webdemo.zip file including the project will be downloaded. After extracting the file to an appropriate location, you can proceed to import the project into your IDE of your choice[1].

The sequel assumes Eclipse is being used, but other IDEs should offer corresponding options. Select "Project\Import" and "Existing Maven Projects" in the next window, navigate to the appropriate location in your disk, and import the project.

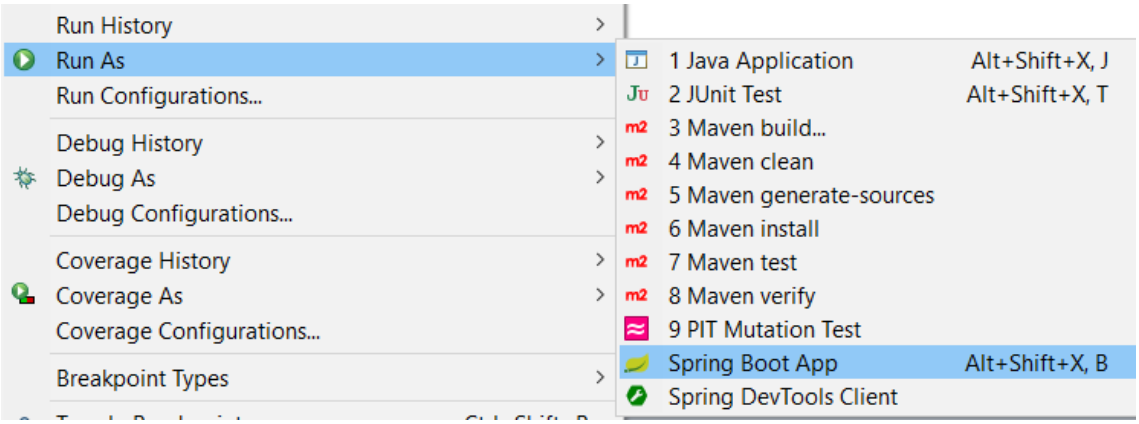


After following this steps, the project should have been imported. It should look something like this:

[1] To do this, you might need to download and install the Maven build system if it isn't already installed in your system (https://maven.apache.org/install.html).



Right click on the project to run it as a Spring Boot app.



A web server instance will start which will run your application and make it available on port 8080 by default. However, the application does not do much yet so you do not have any way to interact with it. Stop the execution of the application before proceeding.

2- Let's now add some things to our project. Add a Greeting class create a Greeting class with a long id and a content string, in the com.example.webdemo package.
Also, create a GreetingController class with the following code:

```java
package com.example.webdemo;

import java.util.concurrent.atomic.AtomicLong;

@RestController
public class GreetingController {

    private static final String template = "Hello, %s!";
    private final AtomicLong counter = new AtomicLong();

    @GetMapping("/greeting")
    public Greeting greeting(@RequestParam(value = "name", defaultValue = "World") String name) {
        return new Greeting(counter.incrementAndGet(), String.format(template, name));
    }
}
```

The @RestController annotation indicates that this class will be responsible for handling web requests. The @GetMapping annotation indicates that the following method should be executed when receiving a request on the indicated endpoint greeting ("/greeting"). Run the app and access localhost:8080/greeting to see the result. Examine to code to understand what is going on.

You can also experiment with providing an additional argument "name" to your url (localhost:8080/greeting?name=John). Examine how this is handled in the code.

In subsequent lessons, we will expand on this project to explore API patterns we will be studying in theoretical classes.